 # Experiment 1:

```
import numpy as np


# Simplified input process


n = int(input("Enter dimension (n) for matrix A and B (n x  n) and vector x (n x 1): "))


A = input("Enter matrix A of size {}. x {}. in a single line row wise elements (separated by space):
".format(n , n))

A = np.matrix(A).reshape(n,n)

print("\nMatrix A:\n", A)


B = input("Enter matrix B of size {}. x {}. in a single line row wise elements (separated by space):
".format(n , n))

B = np.matrix(B).reshape(n,n)

print("\nMatrix B:\n", B)


x = input("Enter Vector x of size {}. x 1 in a single line row wise elements (separated by space):
".format(n))

x = np.matrix(x).reshape(n,1)

print("\nVector x:\n", x)


# Transpose

print("\nTranspose of A:\n", A.T)

print("\nTranspose of B:\n", B.T)


# Matrix multiplication

AB = A @ B

print("\nMatrix AB:\n", AB)


# Verify (AB)^T = BTAT

print("\nVerify (AB)^T = B^T A^T:", np.allclose(AB.T, B.T @ A.T))
```

```
# |A| and |B| and singularity check
det_A = np.linalg.det(A)
det_B = np.linalg.det(B)
print(f"\nDeterminant of A: {det_A}")
print(f"Determinant of B: {det_B}")
print(f"A is {'singular' if det_A == 0 else 'non-singular'}")
print(f"B is {'singular' if det_B == 0 else 'non-singular'}")

# Inverse and verification
if det_A != 0:
    A_inv = np.linalg.inv(A)
    print("\nInverse of A:\n", A_inv)
    print("Verify AA^-1 = I:", np.allclose(A @ A_inv, np.eye(n)))
    print("Verify A^-1A = I:", np.allclose(A_inv @ A, np.eye(n)))
else:
    print("\nA is singular, inverse not possible.")

if det_B != 0:
    B_inv = np.linalg.inv(B)
    print("\nInverse of B:\n", B_inv)
else:
    print("\nB is singular, inverse not possible.")

# Trace
print(f"\nTrace of A: {np.trace(A)}")
print(f"Trace of B: {np.trace(B)}")
if det_A != 0:
    print(f"Trace of A^-1: {np.trace(A_inv)}")
if det_B != 0:
    print(f"Trace of B^-1: {np.trace(B_inv)}")
```

```
print(f"Trace of A^T: {np.trace(A.T)}")
print(f"Trace of B^T: {np.trace(B.T)}")

# Trace verification
print(f"\nVerify trace(AB) = trace(BA): {np.trace(A @ B) == np.trace(B @ A)}")
print("\nVerify trace(A^T B) = trace(A B^T) = trace(B A^T) = trace(B^T A):",
np.trace(A.T @ B) == np.trace(A @ B.T) == np.trace(B @ A.T) == np.trace(B.T @ A))

# Vector operations
y = A @ x
print("\ny = Ax:\n", y)

if det_A != 0:
    x_retrieved = A_inv @ y
    print("Retrieved x from y:\n", x_retrieved)

# Inner product, norm, and orthogonality
inner_product = x.T @ y
print(f"\nInner product of x and y: {inner_product}")
print("x and y are orthogonal:" if np.isclose(inner_product, 0) else "x and y are not orthogonal.")
print(f"\nNorm of x: {np.linalg.norm(x)}")
print(f"Norm of y: {np.linalg.norm(y)}")

# Normalize
x_normalized = x / np.linalg.norm(x)
y_normalized = y / np.linalg.norm(y)
print("\nNormalized x:\n", x_normalized)
print("Normalized y:\n", y_normalized)

# CS inequality
```

```
print("\nVerify CS inequality for x and y:",
inner_product <= np.linalg.norm(x) * np.linalg.norm(y))
if abs(inner_product) <= np.linalg.norm(x) * np.linalg.norm(y) :
   print("CS inequality verified")
else :
   print("CS inequality not verified")

# Additional verifications
print("\nVerify x^T y = y^T x:", np.isclose(inner_product, y.T @ x))
print("\nVerify y^T A x = x^T A^T y:",
np.isclose((y.T @ A) @ x, (x.T @ A.T) @ y))

# Eigenvalues and Eigenvectors
eigvals_A, eigvecs_A = np.linalg.eig(A)
eigvals_B, eigvecs_B = np.linalg.eig(B)
print("\nEigenvalues of A:\n", eigvals_A)
print("Eigenvectors of A:\n", eigvecs_A)
print("\nEigenvalues of B:\n", eigvals_B)
print("Eigenvectors of B:\n", eigvecs_B)

# Verify EVD
print("\nVerify EVD for A:", np.allclose(A, eigvecs_A @ np.diag(eigvals_A) @
np.linalg.inv(eigvecs_A)))
print("Verify EVD for B:", np.allclose(B, eigvecs_B @ np.diag(eigvals_B) @ np.linalg.inv(eigvecs_B)))
```

 # Results:

```
Enter dimension (n) for matrix A and B (n x  n) and vector x (n x 1): 3
Enter matrix A of size 3. x 3. in a single line row wise elements (separated by space): 1 2 3 7 5 9 0 3 2

Matrix A:
 [[1 2 3]
```

[7 5 9]

[0 3 2]]

Enter matrix B of size 3. x 3. in a single line row wise elements (separated by space): 2 5 7 9 2 1 2 4 7

Matrix B:

[[2 5 7]

[9 2 1]

[2 4 7]]

Enter Vector x of size 3. x 1 in a single line row wise elements (separated by space): 2 5 1

Vector x:

[[2]

[5]

[1]]

Transpose of A:

[[1 7 0]

[2 5 3]

[3 9 2]]

Transpose of B:

[[2 9 2]

[5 2 4]

[7 1 7]]

Matrix AB:

[[26 21 30]

[77 81 117]

[31 14 17]]

Verify (AB)^T = B^T A^T: True

Determinant of A: 18.000000000000004

Determinant of B: -61.00000000000001

A is non-singular

B is non-singular

Inverse of A:

[[-0.94444444 0.27777778 0.16666667]

[-0.77777778 0.11111111 0.66666667]

[1.16666667 -0.16666667 -0.5   ]]

Verify AA^-1 = I: True

Verify A^-1A = I: True

Inverse of B:

[[-1.63934426e-01 1.14754098e-01 1.47540984e-01]

[1.00000000e+00 6.09268733e-18 -1.00000000e+00]

[-5.24590164e-01 -3.27868852e-02 6.72131148e-01]]

Trace of A: 8

Trace of B: 11

Trace of A^-1: -1.3333333333333333

Trace of B^-1: 0.5081967213114753

Trace of A^T: 8

Trace of B^T: 11

Verify trace(AB) = trace(BA): True

Verify trace(A^T B) = trace(A B^T) = trace(B A^T) = trace(B^T A): True

y = Ax:

[[15]

[48]

[17]]

Retrieved x from y:

[[2.]

[5.]

[1.]]

Inner product of x and y: [[287]]

x and y are not orthogonal.

Norm of x: 5.477225575051661

Norm of y: 53.08483775994799

Normalized x:

[[0.36514837]

[0.91287093]

[0.18257419]]

Normalized y:

[[0.28256656]

[0.90421299]

[0.3202421]]

Verify CS inequality for x and y: [[True]]

CS inequality verified

Verify x^T y = y^T x: [[True]]

Verify y^T A x = x^T A^T y: [[True]]

Eigenvalues of A:

[10.4591777 +0.j -1.22958885+0.45726125j -1.22958885-0.45726125j]

Eigenvectors of A:

[[0.29197884+0.j 0.19971738+0.16430682j 0.19971738-0.16430682j]

[0.90141644+0.j 0.71098706+0.j 0.71098706-0.j]

[0.31968229+0.j -0.6474643 -0.09167122j -0.6474643 +0.09167122j]]

Eigenvalues of B:

[13.02098767 -3.39918433 1.37819666]

Eigenvectors of B:

[[0.61103754 0.48120326 -0.00844115]

[0.55059405 -0.84519896 -0.81334968]

[0.56875242 0.23255568 0.58171388]]

Verify EVD for A: True

Verify EVD for B: True